

WEATHER DATA ANALYSIS

Data Analysis with Python Internship Project

Internship Organization: Cognevance Technologies

Project Level: Level 1 – Easy

Project: Weather Data Analysis

Prepared as a submission-ready project report covering data collection, cleaning, analysis, visualization, monthly comparison, insights, and dashboard development.

1. Project Overview

This project performs weather data analysis using Python. The dataset contains daily weather observations for Hyderabad across multiple months of 2026. Pandas is used for data loading, cleaning, preprocessing and statistical analysis, while Matplotlib is used to create visualizations. An interactive Streamlit dashboard is also included to make the results easier to explore.

2. Objectives

The main objectives are to load and clean weather data, handle missing and incorrect records, analyze temperature, rainfall and humidity trends, create graphical visualizations, compare weather conditions across months, and prepare a concise insights report. These objectives directly align with the Level 1 Weather Data Analysis task requirements.

3. Dataset

The working dataset is the project-prepared **weather_data.csv** file containing daily Hyderabad weather observations for January–December 2026. It includes Date, Location, Temperature, Humidity, Wind Speed, Pressure, Precipitation and Weather condition fields. The dataset intentionally includes a small number of quality issues so that the cleaning workflow can demonstrate handling of duplicates, missing values, invalid measurements and invalid weather labels.

4. Tools & Technologies

Python 3; Pandas for data manipulation; NumPy for numerical processing; Matplotlib for charts; Seaborn available in the project environment; Streamlit for the interactive dashboard; VS Code for development; CSV for the dataset; GitHub for project submission.

5. Data Loading

The Python source code reads `weather_data.csv` with Pandas, converts the Date column into a datetime type, and converts measurement columns into numeric values. Initial inspection uses dataframe previews, information, descriptive statistics and value counts to understand the structure and quality of the data.

6. Data Cleaning & Preprocessing

The preprocessing workflow removes duplicate rows, converts invalid numeric values to missing values, identifies impossible temperatures, invalid humidity percentages, negative rainfall, invalid pressure/wind values and unsupported weather labels. Numeric missing values are filled using monthly medians followed by the overall median when necessary. Missing weather labels are filled with the mode. Invalid dates are removed. Month and Month_Name columns are then created for monthly comparison.

7. Exploratory Data Analysis

After cleaning, the data is summarized using mean, minimum, maximum and total values. Weather-condition frequencies are also calculated. The analysis focuses on three required weather indicators—temperature, rainfall and humidity—and extends the analysis with wind speed, pressure and weather-condition distribution.

8. Temperature Analysis

The cleaned dataset has an average temperature of **28.66 °C**. The highest recorded/cleaned temperature is **38.00 °C** on **2026-05-02**, while the lowest is **21.00 °C** on **2026-01-04**. The daily temperature chart helps identify variation over the year, while the monthly chart supports direct comparison between months.

9. Humidity Analysis

Average humidity is **57.33%**. The daily humidity visualization shows changes over the observation period and the monthly summary provides a month-by-month comparison. Missing humidity values were handled during preprocessing rather than being left as incomplete records.

10. Rainfall Analysis

Total precipitation in the cleaned dataset is **418.40 mm**. The highest daily rainfall value is **9.40 mm** on **2026-07-04**. Monthly rainfall totals are visualized to make wetter and drier months easy to compare. Negative rainfall values were treated as invalid during cleaning.

11. Monthly Weather Comparison

The dataset is grouped by calendar month to calculate average temperature, average humidity and total rainfall. This allows the project to compare weather conditions across January through December rather than relying only on daily observations. The monthly comparison charts provide a compact view of seasonal variation.

12. Data Visualizations

The report includes five core visualizations: daily temperature trend, daily humidity trend, monthly rainfall, monthly average temperature and weather-condition distribution. These charts support the numerical analysis and make patterns easier to communicate.

13. Interactive Dashboard

The Streamlit dashboard provides date-range filtering and weather-condition filtering, key performance metrics, temperature and humidity trend charts, monthly comparison charts, a weather-condition pie chart, automated insights, a data table and a CSV download option. The dashboard can be launched with the command: **streamlit run dashboard.py**.

14. Key Insights

- Average temperature: **28.66 °C**.
- Average humidity: **57.33%**.
- Total rainfall: **418.40 mm**.
- Highest temperature: **38.00 °C**.
- Lowest temperature: **21.00 °C**.
- Highest rainfall event: **9.40 mm**.
- Most common weather condition: **Cloudy**.

These results demonstrate how Python can convert raw weather records into interpretable trends and insights.

15. Results

The project successfully demonstrates the complete analysis workflow: dataset preparation, Pandas-based cleaning, statistical analysis, visualization, monthly comparison and interactive reporting. The final output consists of reusable source code, raw and cleaned datasets, charts, a README and this project report.

16. Conclusion

Weather Data Analysis using Python provides a practical example of an end-to-end data-analysis workflow. Pandas simplifies data cleaning and aggregation, Matplotlib communicates trends visually, and Streamlit turns the analysis into an interactive dashboard. The project meets the requested analytical goals and provides a foundation for further forecasting and real-time weather analytics.

17. Future Scope

Future improvements can include using a larger live/public weather dataset, adding weather forecasting models, anomaly detection, correlation analysis, automated data ingestion from an API, real-time dashboard refresh, geographic comparisons across cities and deployment of the Streamlit dashboard online.

18. Project Files & GitHub Submission

Recommended repository name: **cognevance_weatherDataAnalysis**. The repository should contain weather_data.csv, cleaned_weather_data.csv, weather_analysis.py, dashboard.py, README.md, Project_Report.pdf, generated chart images and relevant screenshots. The final GitHub repository link can then be submitted according to the internship instructions.

19. Data Quality & Cleaning Summary

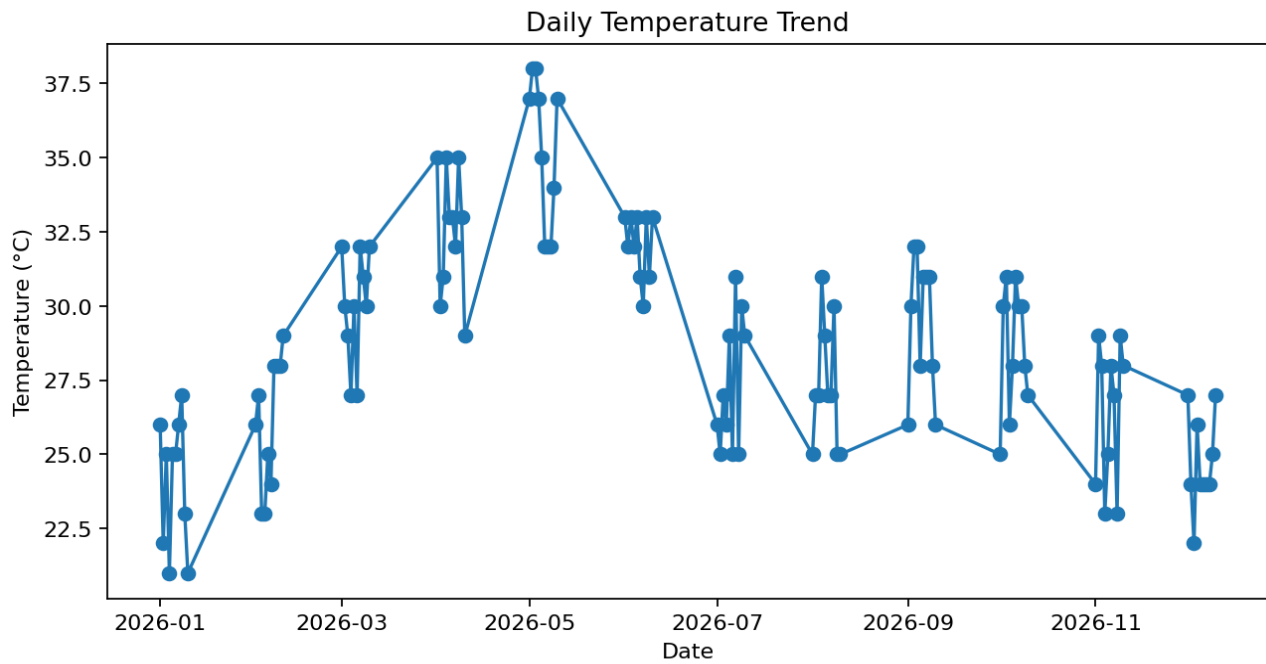
Item	Result / Action
Initial rows	121
Duplicate rows detected	1
Duplicate handling	Duplicate records removed
Invalid temperature	Converted to missing and imputed
Invalid humidity	Converted to missing and imputed
Negative rainfall	Converted to missing and imputed
Invalid weather label	Converted to missing and filled with mode
Missing numeric values	Monthly median, then overall median
Date handling	Converted to datetime; invalid dates removed

20. Task Completion Mapping

Required Task	Implemented
Collect/download weather dataset	Weather dataset prepared and stored as weather_data.csv; public-source replacement can
Load and clean using Pandas	Completed in weather_analysis.py
Handle missing and incorrect records	Completed using validation, imputation and duplicate removal
Analyze temperature, rainfall and humidity	Completed with statistics and trend charts
Create graphs	Completed with Matplotlib
Compare conditions across months	Completed with monthly aggregation and charts
Prepare short insights report	Completed in this PDF and automated dashboard insights

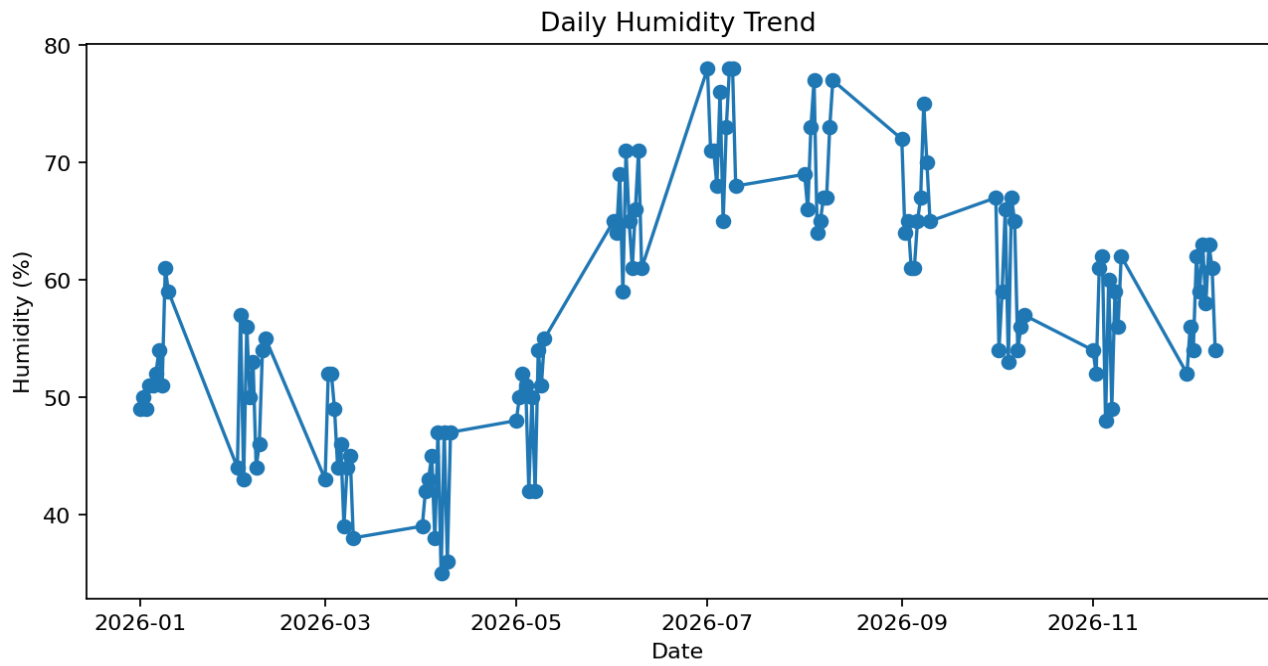
Note on data source: The current weather_data.csv was prepared for this project workflow rather than downloaded from Kaggle. If the internship evaluator strictly requires a Kaggle/online dataset, replace the raw CSV with a sourced public dataset and update the source attribution in README.md and this report before submission.

21. Temperature Trend



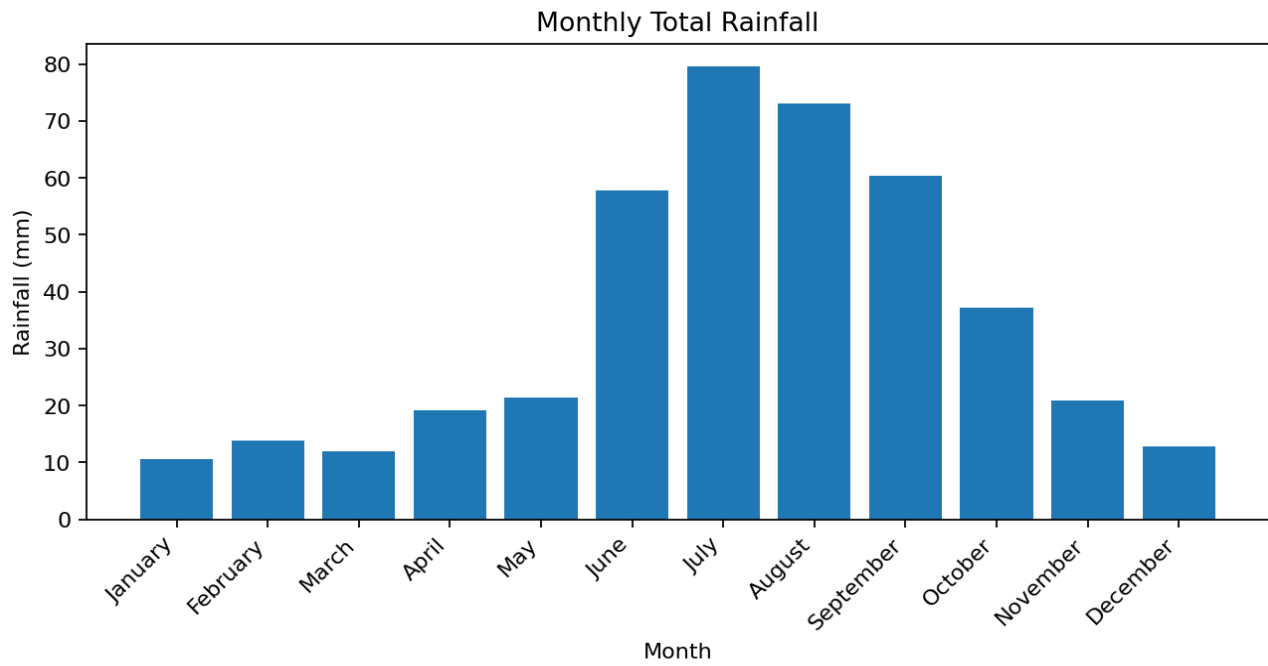
Generated from the cleaned project dataset using Python and Matplotlib.

22. Humidity Trend



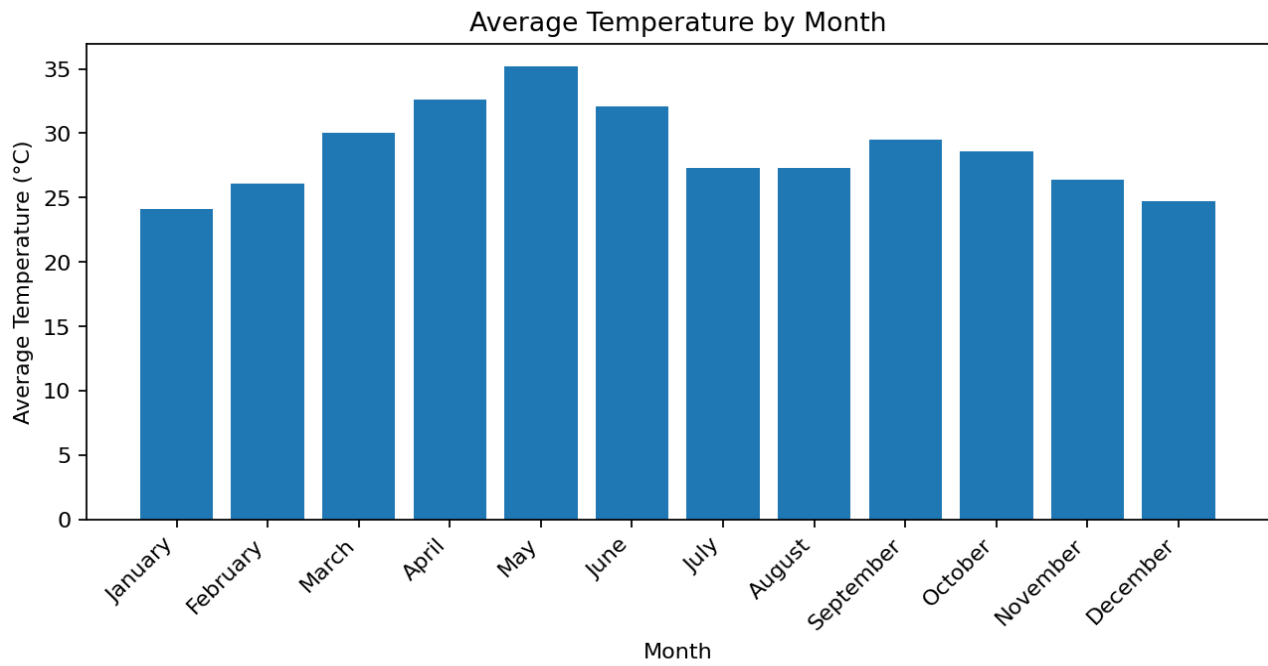
Generated from the cleaned project dataset using Python and Matplotlib.

23. Monthly Rainfall Comparison



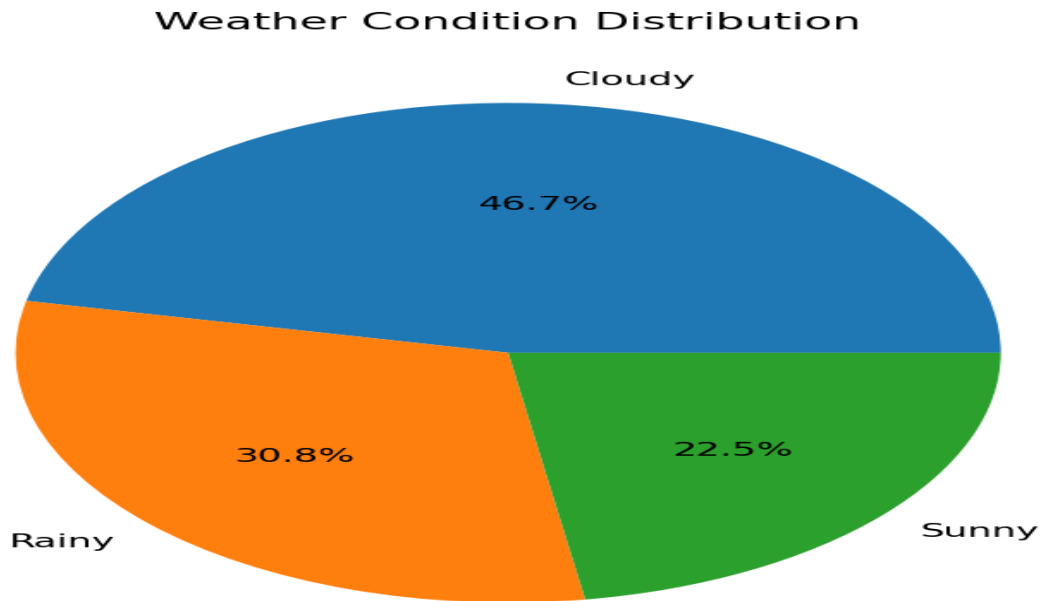
Generated from the cleaned project dataset using Python and Matplotlib.

24. Monthly Temperature Comparison



Generated from the cleaned project dataset using Python and Matplotlib.

25. Weather Condition Distribution



Generated from the cleaned project dataset using Python and Matplotlib.